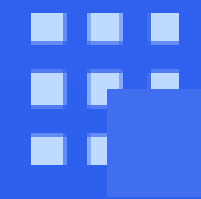


02

接口测试

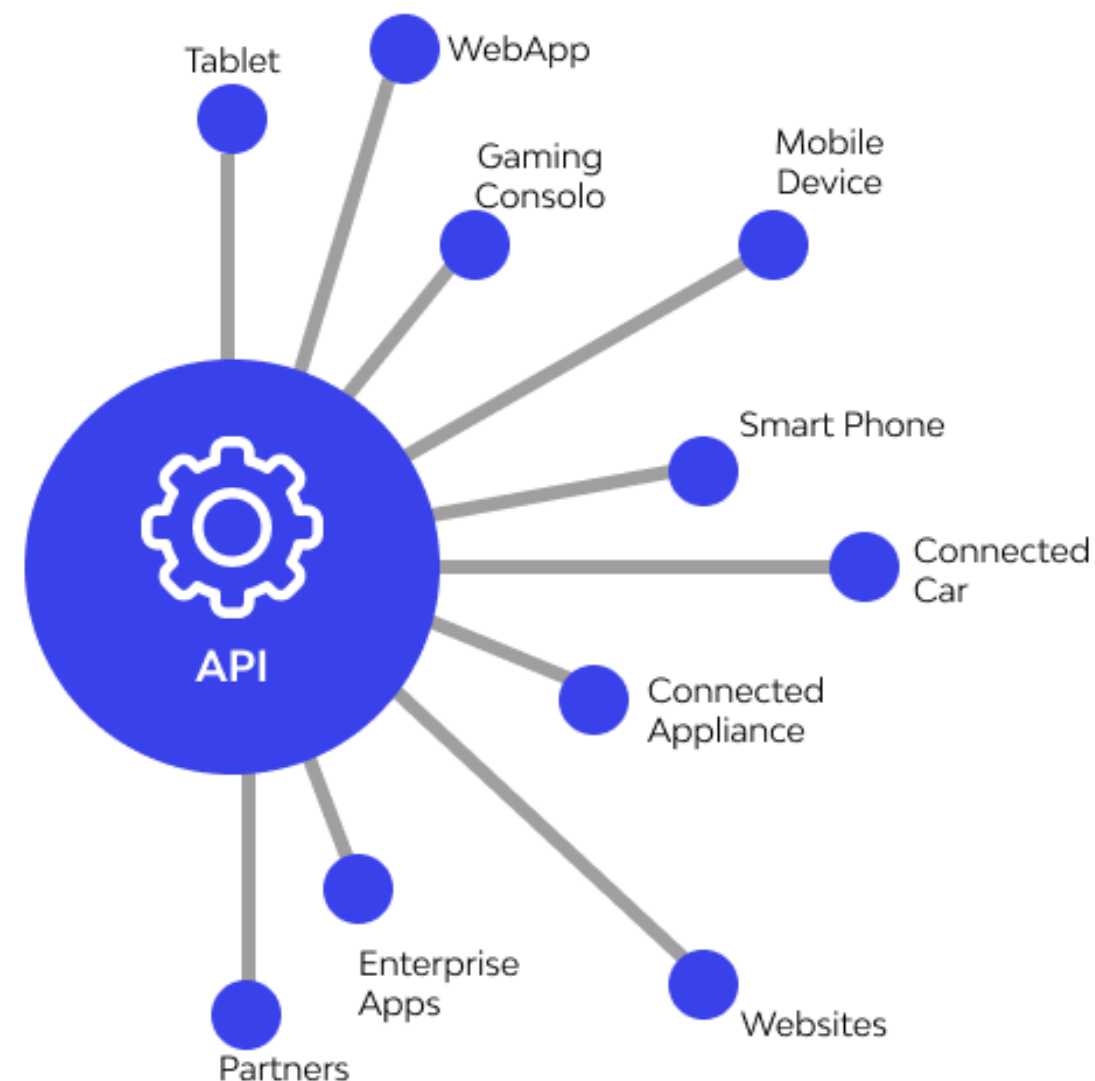


什么是接口测试？

接口测试是指针对软件系统中的接口进行的测试，以验证接口是否能够按照预期进行通信和数据交换。

接口测试的定义

应用程序编程接口测试（简称 API 测试或接口测试）在消息层进行，通常作为集成测试的一部分，用于确定 API 是否满足测试人员对功能、可靠性、性能和安全性的期望。



API测试的重要性

API测试的重要性

- 敏捷开发的关键环节：API 测试提供即时反馈，帮助开发人员快速迭代和改进代码。
- 保障软件质量：通过测试发现和修复缺陷，确保软件的稳定性和可靠性。
- 提升开发效率：早期发现和修复缺陷，避免问题扩大，提高开发效率。
- 适应现代软件开发模式：在微服务架构和敏捷开发模式下，API 测试变得尤为重要。

思考：

- API测试的优势有哪些？
- API测试与UI功能测试有哪些区别？



API测试执行

API测试常见任务

- 功能测试：验证 API 的功能是否符合预期，并针对不同的输入值返回正确的响应。
- 错误处理和输入验证：测试 API 是否能够正确处理错误情况和非法输入。
- 安全测试：测试 API 是否存在安全漏洞，例如 SQL 注入、XSS 攻击等。
- 性能测试：评估 API 的性能指标，例如响应时间、吞吐量、并发处理能力等。
- 兼容性测试：验证 API 是否兼容不同的平台和操作系统。
- 可扩展性测试：评估 API 的可扩展性，确保系统能够处理大量请求。

思考：

- API测试有哪些执行方式？
- API测试的目标是什么？

API功能测试概述

API功能测试概念

功能测试是最基本的接口测试类型，主要目标是验证接口的功能是否符合预期，并能够正确处理输入和输出。

示例：

假设有一个天气查询接口，其功能是接收一个城市名称作为输入，返回该城市当前的天气情况。

思考：

- 如何执行API功能测试？
- API测试应该考虑哪些因素？



API性能测试概述

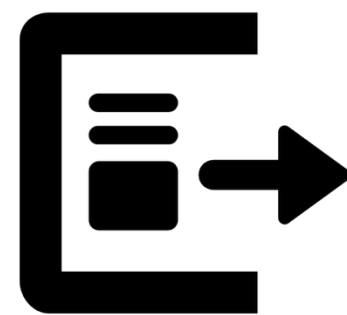
什么是API性能测试?

性能测试是评估接口在实际运行中的性能指标，确保其满足预期。性能测试通常使用工具（如 JMeter）模拟高负载环境，评估接口的稳定性和可靠性。

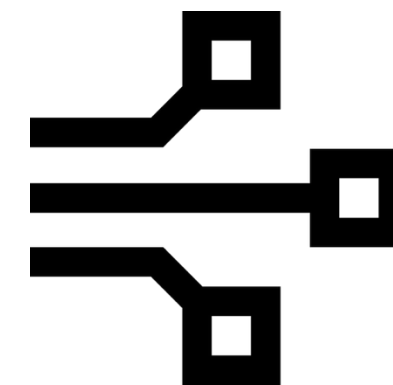
性能测试主要指标



- 响应时间： 接口处理请求并返回结果所需时间，影响用户体验。



- 吞吐量： 单位时间内接口处理请求的数量，影响系统处理能力。



- 并发处理： 接口同时处理多个请求的能力，影响系统在高负载下的性能。

接口安全性测试概述

接口安全性目标

接口安全性测试是通过安全测试工具模拟攻击来验证软件系统接口能否防御非法侵入和攻击的测试。它确保接口不会成为安全漏洞，保障系统的安全性和可靠性。

接口安全性测试的常见攻击类型

- SQL 注入
- 跨站脚本XSS 攻击
- 跨站请求伪造（CSRF）会话劫持

思考：

- 还有哪些常见的攻击类型
- API安全测试主要测试哪些内容

接口兼容性测试概述

接口兼容性目标

兼容性测试则是为了确保接口在各种设备、操作系统、数据库、浏览器或网络环境等条件下，都能正确地执行功能。

接口兼容性测试示例

一个 Web 应用的 API，它被设计为在各种浏览器环境中工作，包括 Google Chrome、Mozilla Firefox、Safari 以及 Microsoft Edge 等。



接口数据一致性测试概述

接口数据一致性的测试目标

接口数据一致性的主要目标是确保数据在传输过程中保持一致性，无论数据是否经过多个接口或系统处理。这种测试确保接口在收到或发送数据时，不会出现数据丢失、数据错误或数据不同步的问题。

接口数据一致性示例

假设在线购物应用中，用户下订单时，订单数据（如商品ID、数量、价格、用户ID）在传输到后端服务器过程中完整无误。同时，服务器返回的订单确认信息（如订单ID、订单状态）也与保存的数据一致。



接口监视概述

什么是接口监视？

接口监视是一种持续的、动态的过程，主要关注接口在实际运行中的行为和性能。这主要包括接口的响应时间、错误率、吞吐量等关键性能指标。接口监视的目的是确保接口始终处于良好的运行状态，并在出现问题时能够立即发现并采取相应的措施。

接口监视示例

在电商平台中，可以实时监控用户下单接口的响应时间和错误率，以快速发现性能问题或后端服务故障，并及时通知相关人员处理，最小化对用户体验的影响。

API 覆盖率分析概述

什么是API 覆盖率分析?

API 覆盖率分析是评估测试效果的一种重要方法，它关注的是在测试过程中 API 的调用是否全面，是否所有的 API 都被充分的测试到。覆盖率高意味着测试的全面性和有效性更高，可能出现的问题被发现和解决的概率也更大。

API覆盖分析示例

假设有一个天气查询的 Web API，在测试天气查询 Web API 时，需确保所有接口（如当前天气、未来天气预报、历史天气记录）都被充分测试，包括正常、异常和边界情况。通过创建测试用例列表并标记已测试的用例，可以有效地管理和跟踪测试进度，确保测试的全面性。

Web 应用 API 分类

Web应用API类别有哪些？

Web 应用 API 主要分为 SOAP 和 REST 两大类。

- SOAP（简单对象访问协议）是 W3C 标准定义的协议，用于 Web 服务请求和响应；
- REST（表述性状态传输）是基于 HTTP 的架构，没有官方标准。

API的用途

API有助于定义验证方法。API 通常用于操作资源，包括读取、创建、更新和删除。了解 API 用途有助于准备测试数据，并定义验证方法。

blog/posts : Operations related to blog posts

GET	/blog/posts/	Returns list of blog posts
POST	/blog/posts/	Creates a new blog post
DELETE	/blog/posts/{id}	Deletes a blog post with mathcing "id"
GET	/blog/posts/{id}	Returns a blog post with matching "id"
PUT	/blog/posts/{id}	Updates a blog post with matching "id" and "checksum"

API响应状态码

什么是API响应状态码

API 响应状态码是 HTTP 协议中用于表示请求处理结果的代码。它由三位数字组成，第一位数字表示响应类别，后两位数字表示具体的响应状态。

响应状态码的表示

1xx: 表示请求已被接收，需要继续处理。

2xx: 表示请求已成功处理。

3xx: 表示请求已被重定向。

4xx: 表示客户端错误，例如请求语法错误或请求的资源不存在。

5xx: 表示服务器错误，例如服务器内部错误或服务器暂时不可用。

API 的实际响应状态码由开发团队指定。测试人员需要验证这些状态码是否符合全球标准和需求文档中的规定。

简单API

什么是简单API?

在测试项目中，总有一些 API 比较简单，只有一两个输入，例如登录 API、获取Token API、健康检查 API 等。

简单API的测试方法

- 关键 API 的优先级：简单但关键的 API（如登录、获取 Token、健康检查）应优先测试，以确保系统的基础功能正常。
- 避免混合测试：避免在一个测试用例中测试多个 API，以简化错误调试。
- 端到端测试的时机：端到端测试应在所有 API 单独测试完成后进行。

多个API的处理策略

如何处理复杂API?

- 组织 API: 建议将 API 分类, 便于测试管理, 提高测试覆盖率和集成度。
- 共享信息: 同类 API 共享资源类型、路径等通用信息。
- 测试可重用性: 使用统一结构组织测试, 提高测试可重用性和扩展性。
- 自动化测试: 利用自动化加快测试流程, 保存测试数据和执行历史。
- API 更改管理: API 更改需明确要求, 测试人员需保持警惕并及时调整。
- 测试速度: API 测试速度快于系统级测试。
- 黑盒测试: API 测试属于黑盒测试, 验证输入输出。
- 数据驱动自动化: 数据驱动方法提高 API 测试覆盖率。
- 测试脚本重复使用: 测试脚本可重复使用, 提高测试效率。
- 早期测试: API 测试可在软件开发生命周期早期进行。
- 模拟技术: 模拟技术帮助在开发实际 API 前验证 API 及其集成。

API响应

什么是API响应?

```
Host: example.com
Content-Type: application/json
Authorization: Bearer your_access_token
```

API 响应是指当客户端（如浏览器、移动应用等）向服务器发送请求时，服务器返回给客户端的数据。它通常包括两部分：状态码和响应正文。API 响应内容因其数据类型和大小而异，可以是纯文本、JSON 文件、XML 文档等。

```
{
  "username": "john_doe",
  "email": "john@example.com",
  "password": "secure_password"
}
```

验证 API 响应正文内容的方法

- 比较整个响应：适用于具有静态内容的简单响应。
- 比较每个属性值：适用于 JSON 或 XML 格式的响应，验证动态内容或个体值。
- 正则表达式匹配：用于验证具有特定模式的数据响应，处理复杂的动态数据。

正向和负向测试

正向和负向测试概述

API 测试需要进行正向和负向测试，以确保 API 正常工作。由于 API 测试被认为是一种黑盒测试，因此两种类型的测试都是由输入和输出数据驱动的。

正向和负向测试方法

- 验证 API 是否接收输入并返回要求中指定的预期输出。
- 验证响应状态代码是否按照要求返回，无论是返回 2xx 还是错误代码。指定具有最少必填字段和最多字段的输入。当预期输出不存在时，验证 API 是否返回适当的响应。
- 执行输入验证测试，使用不同级别的授权验证 API 的行为。

手工API测试现状

手工API测试有哪些痛点

- 测试数据管理和验证的复杂性：随着测试深入，任务复杂，选择合适的测试数据和验证方法至关重要。
- 逐个验证 JSON/XML 数据的困难：验证 JSON/XML 数据或使用对象映射选择正确方法困难。
- 自动化测试的结构要求：自动化测试结构应可扩展、可重用和可维护。

思考：

- 自动化API测试有哪些优势？

— Required Parameters —

q	STRING	Hello, world!	REQUIRED The input text to translate. Repeat this parameter to perform translation operations on multiple text inputs.
target	STRING	zh-CN	REQUIRED The language to use for translation of the input text, set to one of the language codes listed in the overview tab

人工构造、
数据库中获取

从其他请求中
获取

自动化API测试概述

什么是自动化API测试？

自动化测试是一种使用测试工具或脚本来自动执行测试用例的方法。在接口测试中，自动化测试极大地提高了测试的效率和准确性。

自动化API测试与手工API测试区别

对于天气查询接口，手工测试需要逐一输入城市名称并检查结果，但自动化测试可以使用工具如 Postman 或 JMeter，编写脚本自动发送查询请求并验证结果。这种方法节省人力，特别适用于大量测试。自动化测试还便于回归测试，确保接口改动后功能未受影响。

```
OkHttpClient client = new OkHttpClient();

MediaType mediaType = MediaType.parse("application/x-www-form-urlencoded");
RequestBody body = RequestBody.create(mediaType, "q=Hello%2C%20world!&target=zh-CN&source=en");
Request request = new Request.Builder()
    .url("https://google-translate1.p.rapidapi.com/language/translate/v2")
    .post(body)
    .addHeader("content-type", "application/x-www-form-urlencoded")
    .addHeader("Accept-Encoding", "application/gzip")
    .addHeader("X-RapidAPI-Key", "e58372a891mshdac89d87de48f52p1cb97fjsnb712bfe3de4a")
    .addHeader("X-RapidAPI-Host", "google-translate1.p.rapidapi.com")
    .build();

Response response = client.newCall(request).execute();
```

Java API测试库 - OkHttp

```
import requests

url = "https://google-translate1.p.rapidapi.com/language/translate/v2"

payload = {
    "q": "Hello, world!",
    "target": "zh-CN",
    "source": "en"
}
headers = {
    "content-type": "application/x-www-form-urlencoded",
    "Accept-Encoding": "application/gzip",
    "X-RapidAPI-Key": "e58372a891mshdac89d87de48f52p1cb97fjsnb712bfe3de4a",
    "X-RapidAPI-Host": "google-translate1.p.rapidapi.com"
}

response = requests.post(url, data=payload, headers=headers)

print(response.json())
```

Python API测试库 - requests

持续集成概述

什么是持续集成？

持续集成（CI）是一种软件开发实践，通过频繁合并代码到主分支来尽早发现和修复集成错误，提高软件质量。在 CI 流程中，接口测试是关键步骤之一，每次代码提交都会触发构建和测试过程。如果任何步骤失败，CI 系统会立即通知开发人员，以便在早期阶段解决问题，提高开发效率。

持续集成示例

开发在线购物平台时，可以使用自动化测试工具来执行针对 REST API 接口的测试。如果测试失败，CI 系统会立即停止构建过程，并通知开发人员，帮助他们快速定位并修复问题。这确保了 API 接口始终可用且符合预期行为，提高了软件产品的质量和可靠性。

API 测试工程实践经验

- 自动化 API 测试：使用专门的软件工具自动创建和执行 API 测试，提高测试效率和准确性。
- 整个软件生命周期中进行接口测试：在开发到部署的整个过程中测试 API，及早发现问题并确保 API 在不同环境中正常工作。
- 编写可重用的子测试：创建可重用于多个 API 的模块化测试脚本，节省时间并提高效率，同时确保不同 API 之间的测试保持一致。